Data Structures in Java

Lecture 3: More Big-O. Recursion.
Proofs by Induction.

9/14/2016

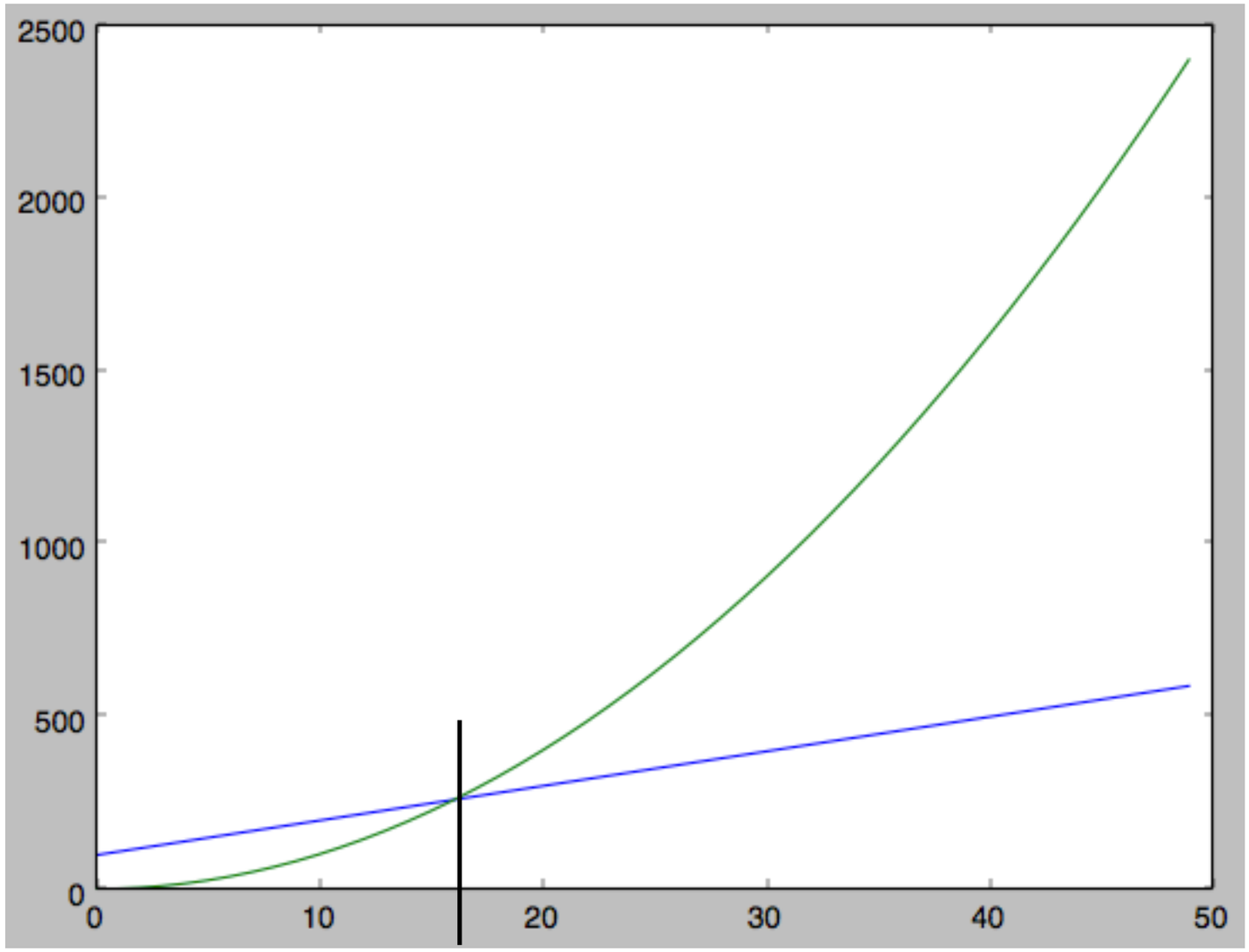
Daniel Bauer

Comparing Function Growth: Big-O Notation

$T(N) = O(f(N))$ if there are positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.

"T(N) is in the order of f(N)"

"f(N) is an upper bound on T(N)"

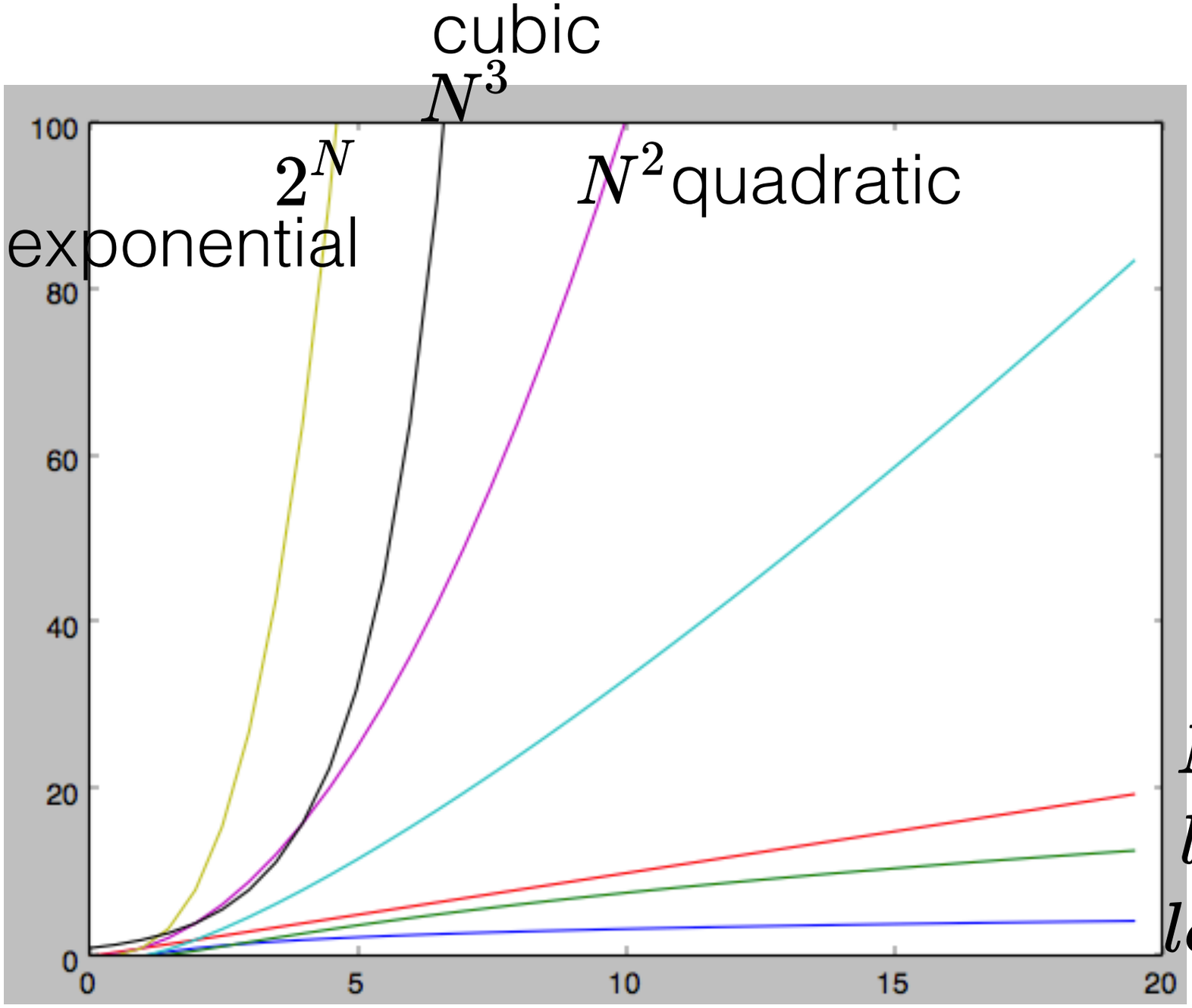


f(N) = N² + 2

T(N) = 10N+ 100

e.g. $c = 1$, $n_0 = 16.1$

Typical Growth Rates



N	linear
$log^2(N)$	log-squared
$logN$	logarithmic
c	constant

Comparing Function Growth: Additional Notations

- Upper Bound:

$T(N) = O(f(N))$ if there are positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.

- Lower Bound:

$T(N) = \Omega(f(N))$ if there are positive constants c and n_0 such that $T(N) \geq cf(N)$ when $N \geq n_0$.

- Tight Bound: T(N) and f(N) grow at the same rate

$T(N) = \Theta(f(N))$ if $T(N) = \Omega(f(N))$ and $T(N) = O(f(N))$.

Rules for Big-O (1)

If $T(N)$ is a polynomial of degree k then

$$T(N) = \Theta(N^k)$$

For instance: $9N^3 + 12N^2 - 5 = \Theta(N^3)$

$log^k(N) = (log(N))^k = O(N)$ for any k .

$log_a(N) = \Theta(log_2(N))$ for any a .

Rules for Big-O (2)

If $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$ then

1. $T_1(N) + T_2(N) = O(f(N) + g(N))$
   $= O(\max(f(N), g(N))$

2. $T_1(N) * T_2(N) = O(f(N) * g(N))$

General Rules: Basic *for*-loops

Compute $\sum_{i=1}^{N} i^3$

```
public static int sum(int n){
  int partialSum = 0;
  for (int i = 1; i <= n; i++)
      partialSum += i * i * i;
  return partialSum;
}
```

1 step (initialization)

+1 step for last test

1 step

N iterations

2 steps each

4 steps each

1 step

$$T(N) = 6N + 4 = O(N)$$

(running time of statements in the loop) X (iterations)

If loop runs a constant number of times: *O(c)*

Generally, we do not need to count individual steps!

General Rules: Nested *L*oops

Analyze inside-out.

```
for (i=0; i < n; i++)
  for (j=0; j < n; j++)
    k++;
```

N iterations $N \cdot O(N) = O(N^2)$

N iterations $O(N)$

1 step each $O(c)$

General Rules: Consecutive Statements

```
for (i = 0; i < n; i++)
  a[i] = 0;
```

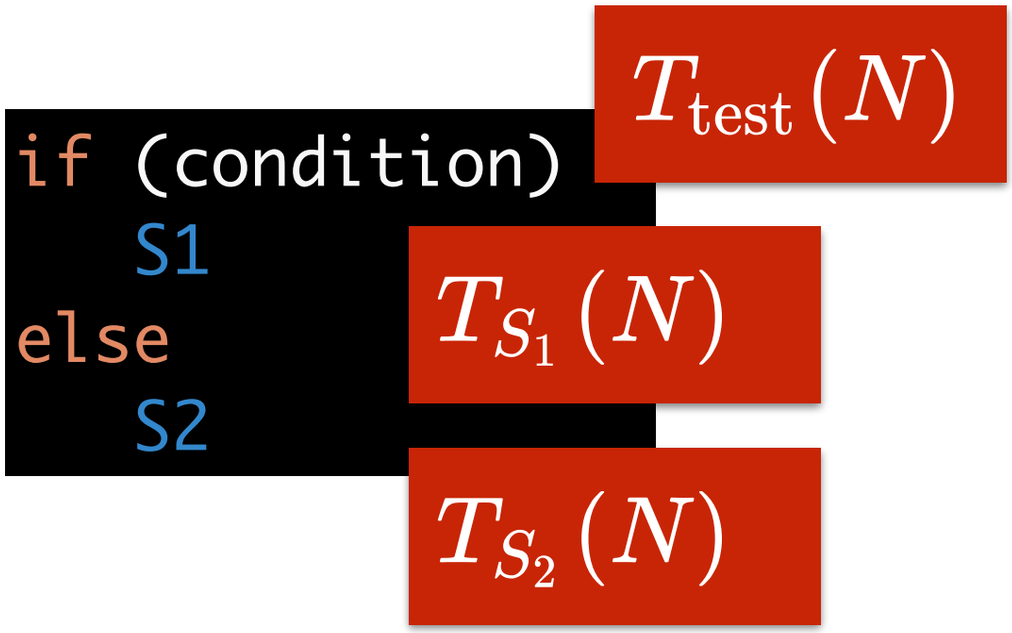
$O(N)$

```
for (i=0; i < n; i++)
  for (j = 0; j < n; j++)
      a[i] += a[j] + i + j;
```

$O(N^2)$

$$O(N) + O(N^2) = O(N^2)$$

General Rules: *if/else* conditionals



$$T(N) = O(\max(T_{S_1}(N), T_{S_2}(N)) + T_{\text{test}}(N))$$

General Rules: calling methods

```
for (int i=0; i < n; i++) {
    someMethod(a[i]);
}
```

N steps

$T_{someMethod}(M)$

$$T(N) = N \cdot T_{someMethod}(M)$$

Logarithms in the Running Time

```
public class BinarySearch {

        public int binarySearch(Integer[ ] a, Integer x ) {
                int low = 0, high = a.length - 1;

                while( low <= high ) {
                    int mid = ( low + high ) / 2;

                    if( a[ mid ] < x )
                        low = mid + 1;
                    else if( a[ mid ] > x )
                        high = mid - 1;
                    else
                        return mid;   // Found
                }
                return -1;
        }
}
```

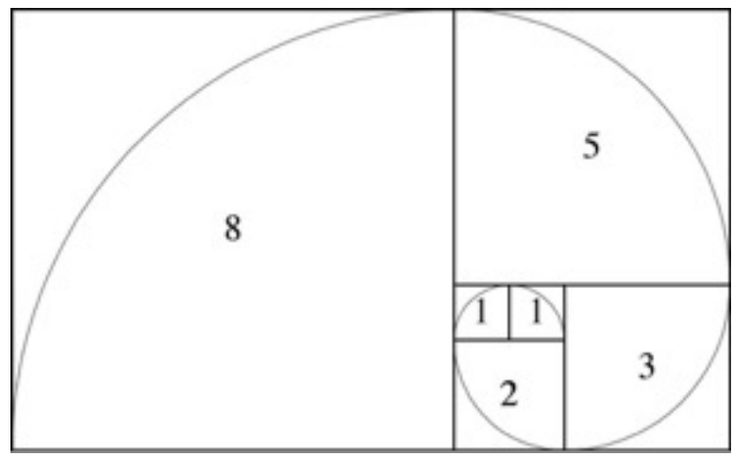
$log_2(N)$

$O(1)$

Each iteration of the while loop cuts remaining partition in half.
There are $\log_2(N)$ iterations. The total runtime is $log_2(N) \cdot O(1) = O(\log N)$.

Fibonacci Sequence

- 1, 1, 2, 3, 5, 8, 13, 21, …



$$F_1 = 1$$
$$F_2 = 1$$
$$F_{k+1} = F_k + F_{k-1}$$

Recursive Definition

- Closed form solution for F_n is complicated.

- Instead: easier to compute this algorithmically.

Fibonacci in Java

```
public class Fibonacci {

  public static void main(String[] args) {
    Fibonacci fib = new Fibonacci();
    int k =  Integer.parseInt(args[0]);
    System.out.println(fib.fibonacci(k));
  }

  public long fibonacci(int k) throws IllegalArgumentException{
    if (k < 1) {
      throw new IllegalArgumentException("Expecting a positive integer.");
    }
    if (k == 1 | k == 2) {
      return 1;
    } else {
      return fibonacci(k-1) + fibonacci(k-2);
      }
    }
}
```

Base case

Recursive call - making progress

How many steps does the algorithm need?

```
public class Fibonacci {

  public static void main(String[] args) {
    Fibonacci fib = new Fibonacci();
    int k =  Integer.parseInt(args[0]);
    System.out.println(fib.fibonacci(k));
  }

  public long fibonacci(int k) throws IllegalArgumentException{
    if (k < 1) {
      throw new IllegalArgumentException("Expecting a positive integer.");
    }
    if (k == 1 | k == 2) {
      return 1;
    } else {
      return fibonacci(k-1) + fibonacci(k-2);
      }
    }
}
```

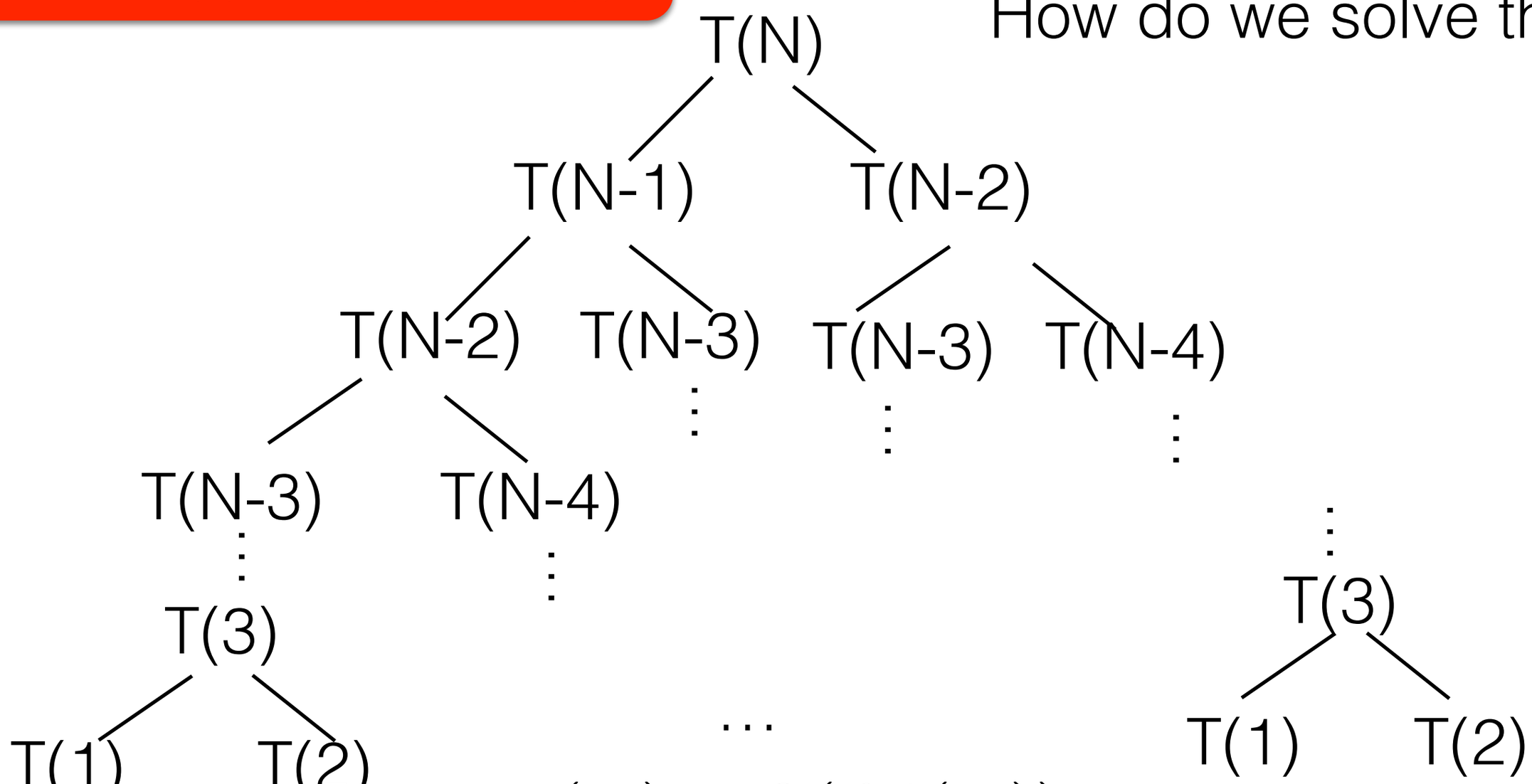
Base case: 1 step T(1) = O(c), T(2) = O(c)

Recursive calls: T(k) = O(T(k-1) + T(k-2))

Analyzing the Recursive Fibonacci Solution

Recursive calls: T(k) = O(T(k-1) + T(k-2))
Base case: T(1) = O(c), T(2) = O(c)

Recurrence Relation.
How do we solve this?



$$T(N) = \Theta(fib(N))$$

Analyzing the Recursive Fibonacci Solution (2)

- We prove that $fib(N) \geq (3/2)^{N-1}$ for any $N \geq 6$
- Base case: $fib(6) = 8 \geq (3/2)^5 = 7.59375$
- Inductive step:
  - Assume the theorem holds for i =6,…,k
  - We need to show that $fib(k+1) \geq (3/2)^k$

Analyzing the Recursive Fibonacci Solution (3)

$$
\begin{aligned}
fib(k+1) &\geq (3/2)^k + (3/2)^{k-1} \\
&\geq (2/3)(3/2)^{k+1} + (2/3) \cdot (2/3) \cdot (3/2)^{k+1} \\
&\geq (4/9 + 2/3)(3/2)^{k+1} \\
&\geq (10/9)(3/2)^{k+1} \\
&\geq (3/2)^{k+1} \quad \square
\end{aligned}
$$

Therefore: $fib(N) = \Omega((3/2)^{N-1}) = \Omega(1.5^{N-1})$

also (from Weiss): $fib(N) = O((5/3)^{N-1}) = O(1.67^{N-1})$

actual tight bound: $fib(N) = \Theta((\frac{1+\sqrt{5}}{2})^{N-1}) \approx \Theta(1.62^{N-1})$

Analyzing the Recursive Fibonacci Solution (3)

$$fib(k+1) = fib(k) + fib(k-1) \geq (3/2)^{k-1} + (3/2)^{k-2}$$

$$= (2/3)(3/2)^{k} + (2/3)(2/3)(3/2)^{k}$$

$$= (10/9)(3/2)^{k}$$

$$\geq (3/2)^{k}$$

□

Therefore: $fib(N) = \Omega((3/2)^{N-1}) = \Omega(1.5^{N-1})$

also (from Weiss): $fib(N) = O((5/3)^{N-1}) = O(1.67^{N-1})$

actual tight bound: $fib(N) = \Theta((\frac{1+\sqrt{5}}{2})^{N-1}) \approx \Theta(1.62^{N-1})$

Four Rules for Recursion

1. B*ase Case*

2. *Making Progress*

3. *Design Rule* - Assume all recursive calls work.

4. *Compound Interest Rules* - Never duplicate work by solving the same instance of a problem in separate recursive calls.

Fibonacci Sequence v.2

```
public long fibonacci(int k) throws IllegalArgumentException{

  if (k < 1) {
    throw new IllegalArgumentException("Expecting a positive integer.");
  }
  long b = 1; //k-2
  long a = 1; //k-1
  for (int i=3; i<=k; i++) {
    long new_fib = a + b;
    b = a;
    a = new_fib;
  }
  return a;
}
```

Dynamic programming: Cache intermediate solutions so they can be re-used.